AI Infrastructure

The vocabulary every later lesson's AI segment assumes instead of re-explaining

dbt MCP server
The access layer that lets an agent see a project's metadata

`uvx dbt-mcp`	Starts self-hosted server
`Self-hosted`	Full command access, v1/v2/platform
`Remote: HTTP, platform-hosted`	Consumption only
`Discovery, Admin, Semantic Layer`	What it exposes

AI skills
A markdown file installed into the agent, not a dbt package

`/plugin marketplace add dbt-labs/dbt-agent-skills`	Add the repo
`/plugin install dbt@dbt-agent-marketplace`	Enable it
`npx skills add dbt-labs/dbt-agent-skills --global`	Other agents
`Analytics engineering`	What the repo covers
`Semantic layer work`	What the repo covers
`Platform ops, v1->v2 migration`	What the repo covers

Context engineering
Deciding what an agent gets to see, written as real dbt code

```
packages:
  - package: dbt-labs/dbt_context_engineering
```

`dbt deps`	Installs it
`Chunking without cutting a sentence`	What it's for
`AI calls, spend ceiling, run log`	What it's for
`Vector search, no index required`	What it's for
`Grounding/accuracy tests`	What it's for

Hooks
The agent's own harness enforces this, nobody has to remember

`.claude/settings.local.json`	Where hooks live
`sqlfmt --diff $CLAUDE_FILE_PATH`	PostToolUse: format on save
`Refuse a write outside models/`	PreToolUse: block it
`Lint the file the agent just wrote`	PostToolUse: every time

A good prompt
Structure every request the same way so nothing gets assumed

`Role`	Senior analytics engineer on Snowflake
`Context`	Which model, what must stay true
`Task`	The specific change, named
`Format`	Where the output goes
`Constraints`	Dialect, fixtures, no warehouse cost

Ask Wizard to plan first
Let it explain the blast radius before it touches a file

```
Review my uncommitted changes. Explain which
dbt resources changed, trace downstream impact,
and propose a validation plan. Do not edit yet.
```

Validation tiers
Choose how much checking an agent's work gets, before it runs

`Light`	Syntax, lint, no materialize
`Medium (default)`	+ compiles, builds in dev
`Heavy`	+ dev/prod diff, row counts
`Skip`	No structured validation

Subagents
Split a task too big for one agent, scoped like RBAC

`Planner`	Breaks the task into pieces
`Generator`	Does the work, clean context
`Evaluator`	Checks compile, tests, conventions
`Each gets only its own scope`	Not the whole project